Leaky Bucket Algorithm — Lab Exam Algorithm

Algorithm

1. **Start** the program.

2. Create a `Scanner` object to read input values from the user.

3. Read the **number of time intervals**, **bucket size**, and **output rate** from the user.

4. Initialize the number of packets currently present in the bucket to **0**.

5. Repeat the following steps for each time interval from **1** to **n**.

6. Read the number of **incoming packets** at the current time interval.

7. Calculate the number of packets lost because of bucket overflow.

```
loss = max(0, packets + incoming - bucketSize)
```

8. Add the incoming packets to the bucket. If the total exceeds the bucket capacity, store only up to the bucket size.

```
packets = min(bucketSize, packets + incoming)
```

9. Calculate the number of packets sent out according to the fixed output rate. The number sent cannot exceed the packets currently in the bucket.

```
sent = min(packets, outputRate)
```

10. Remove the transmitted packets from the bucket.

```
packets = packets - sent
```

11. Display the current time interval, incoming packets, packet loss, packets in the bucket, packets sent out, and remaining packets.

12. Repeat the process for the next time interval.

13. After all time intervals are processed, close the `Scanner`.

14. **Stop** the program.

Easy Memory Order

Input → Initialize → Incoming → Loss → Add → Send → Remove → Display → Repeat → Close

Important Calculations

Packet loss:

```
max(0, packets + incoming - bucketSize)
```

Packets entering bucket:

```
min(bucketSize, packets + incoming)
```

Packets sent:

```
min(packets, outputRate)
```

Basic idea: Packets enter → Overflow is lost → Bucket stores packets up to its capacity → Fixed rate sends packets → Remaining packets stay for the next interval.